

Mulund College of Commerce (Autonomous)

Mulund (W), Mumbai — Affiliated to University of Mumbai | PTVA

T.Y. B.Sc. Information Technology | Semester V

Subject: ASP.NET Core

Assignment 1

Due Date:	17th July 2026	Max Marks:	—
Reference:	Pro ASP.NET Core 6 — Adam Freeman (Apress, 9th Ed.)	Class:	T.Y. B.Sc. IT — Sem V

Q 1. Define application frameworks and utility frameworks used in ASP.NET Core. State the purpose of each.

Answer

Introduction

ASP.NET Core is a modern, open-source, cross-platform web development framework developed by Microsoft. It provides two major categories of frameworks that together simplify the process of building robust, scalable, and maintainable web applications: **Application Frameworks** and **Utility Frameworks**.

Part A — Application Frameworks

An **application framework** is a collection of pre-written code, libraries, and tools that provides a structured, ready-made approach for building software applications. Instead of writing every component from scratch, developers use the framework's built-in features to handle common tasks such as routing, database access, authentication, and user interface development.

ASP.NET Core includes the following application frameworks:

1. MVC (Model-View-Controller)

MVC is a design pattern that separates an application into three distinct components, following the principle of Separation of Concerns:

- **Model** — Represents data and business logic. Communicates with the database and applies business rules. Example: A Student model containing Id, Name, and Age.
- **View** — The user interface. Displays data to users as HTML pages using Razor syntax (.cshtml files).
- **Controller** — The intermediary between Model and View. Receives HTTP requests, fetches data from the Model, and selects the appropriate View to display.

Purpose: Suitable for large, complex web applications requiring clear separation between UI, data, and business logic. Improves maintainability and testability.

```
// Model
public class Student { public int Id { get; set; } public string Name { get; set; } }

// Controller
public class StudentController : Controller {
    public IActionResult Index() { return View(); }
}

<!-- View (Index.cshtml) -->
<h1>Student List</h1>
```

2. Razor Pages

Razor Pages is a page-based framework in ASP.NET Core that combines the user interface and page logic in a single page, making it simpler than MVC for smaller applications. Each Razor Page has two files: a **.cshtml** file for the UI and a **.cshtml.cs** (Page Model) file for the logic.

Purpose: Ideal for simple web applications, data-entry forms, and pages where a full MVC structure would be unnecessarily complex.

```
// Index.cshtml.cs – Page Model
public class IndexModel : PageModel {
    public void OnGet() { /* Runs on GET request */ }
}
```

3. Web API Framework

ASP.NET Core Web API is used to build RESTful services that allow communication between applications over HTTP. Instead of returning HTML pages, Web API returns data in JSON format that can be consumed by mobile apps, SPAs, or third-party systems.

Purpose: Building backend services that supply data to front-end applications, mobile apps, and third-party integrations. Supports HTTP methods GET, POST, PUT, and DELETE.

4. Blazor Framework

Blazor is a modern web UI framework that allows developers to build interactive web applications using **C# instead of JavaScript**. It supports two modes: Blazor Server (logic runs on server via SignalR) and Blazor WebAssembly (C# runs directly in the browser).

Purpose: Building interactive single-page applications (SPAs) and component-based UIs using the existing .NET skillset, eliminating the need for JavaScript frameworks.

Application Frameworks — Comparison

Framework	Returns	Best For	Key Feature
MVC	HTML Pages	Large, complex web apps	Separation of Concerns
Razor Pages	HTML Pages	Simple page-based apps	Page + PageModel together
Web API	JSON / XML	Mobile apps, SPAs	RESTful services over HTTP
Blazor	Interactive UI	SPAs without JavaScript	C# replaces JavaScript

Part B — Utility Frameworks

Utility frameworks are collections of built-in libraries and services in ASP.NET Core that provide common functionality required by applications — such as database access, logging, configuration, and security — without the need to write these features from scratch. They work alongside application frameworks to simplify development and improve maintainability.

1. Dependency Injection (DI) Framework

Automatically provides required services and objects to classes, reducing tight coupling between components. Classes declare what they need through constructor parameters and the DI container supplies them automatically.

Purpose: Promotes loose coupling, makes code easier to test (services can be replaced with mock objects), and improves maintainability.

```
public class HomeController : Controller {
    private readonly ILogger<HomeController> _logger;
    public HomeController(ILogger<HomeController> logger) { _logger = logger; }
}
```

2. Configuration Framework

Manages application settings stored in appsettings.json. Allows developers to store database connection strings, API keys, and custom settings outside the code.

Purpose: Centralises settings management. Supports multiple environments (Development, Production) without changing code.

3. Logging Framework

Records application events, warnings, and errors at different severity levels: Trace, Debug, Information, Warning, Error, and Critical.

Purpose: Monitors application behaviour and helps developers troubleshoot issues in development and production environments.

4. Entity Framework Core (EF Core)

An Object-Relational Mapper (ORM) that allows developers to interact with databases using C# objects instead of writing SQL queries. Operations such as inserting, updating, deleting, and retrieving data are performed through the DbContext class.

Purpose: Eliminates boilerplate SQL code, improves developer productivity, and makes database operations type-safe and testable.

5. Middleware Framework

Middleware components process HTTP requests and responses, forming a pipeline through which every request passes. Each middleware component performs a specific task (e.g., HTTPS redirection, routing, authentication) before passing the request to the next.

Purpose: Provides a modular, flexible mechanism to process requests and responses in a controlled order.

6. Authentication and Authorization Framework

Authentication verifies user identity (Who are you?), while Authorization controls what an authenticated user can access (What can you do?). The [Authorize] attribute restricts access to specific pages or actions.

Purpose: Secures web applications by ensuring only authorised users can access protected resources.

7. Caching Framework

Temporarily stores frequently accessed data in memory, reducing the need for repeated database calls. Supports in-memory caching and distributed caching.

Purpose: Significantly improves application performance and reduces database load for frequently requested data.

Utility Frameworks — Summary Table

Utility Framework	Purpose	Key Keyword
Dependency Injection	Auto-provides services — reduces coupling	Constructor Injection
Configuration	Manages settings from appsettings.json	builder.Configuration
Logging	Records events, warnings, errors	_logger.LogInformation()
Entity Framework Core	Database access using C# objects (ORM)	DbContext, DbSet
Middleware	Processes HTTP request/response pipeline	app.Use...()
Authentication & Auth	Verifies identity and controls access	[Authorize]
Caching	Stores data in memory for fast retrieval	_cache.Set()

Reference: Freeman, A. (2022). *Pro ASP.NET Core 6 (9th ed.)*. Apress. — Chapters 1–5

Q Use the NuGet Package Manager to add and update packages in an ASP.NET Core project and
2. explain its role in application development.

Answer

What is NuGet Package Manager?

NuGet is the official package manager for the .NET ecosystem. It allows developers to discover, install, update, and remove third-party libraries and tools — called **NuGet packages** — that add additional functionality to ASP.NET Core applications. NuGet automates the process of downloading and integrating external libraries, eliminating the need to manually manage DLL files.

What is a NuGet Package?

A NuGet package is a compressed archive (.nupkg) that contains:

- **DLL files** — compiled, ready-to-use library code
- **Metadata** — package name, version number, author, description
- **Dependencies** — other packages this package depends on
- **Version information** — for compatibility and upgrade management

Role of NuGet in ASP.NET Core Application Development

Role	Explanation
Extends Functionality	Adds features like EF Core, authentication, JSON handling without writing from scratch
Dependency Management	Automatically resolves and installs all packages that a library depends on
Version Control	Tracks exact versions used — ensures consistent builds across machines
Easy Updates	Single command updates a library to the latest or a specific version
Project Portability	Package references stored in .csproj — dotnet restore rebuilds on any machine
Official & Community Libs	Access to both Microsoft-official and community-contributed packages

Method 1 — Adding Packages Using Visual Studio

Step-by-step process in Visual Studio:

Step	Action
Step 1	Right-click the project in Solution Explorer
Step 2	Select "Manage NuGet Packages"
Step 3	Click the "Browse" tab and search for the package
Step 4	Select the package from results and choose the version
Step 5	Click "Install" — Visual Studio downloads and adds the reference

Method 2 — Adding Packages Using .NET CLI

```
# Install Entity Framework Core for SQL Server
dotnet add package Microsoft.EntityFrameworkCore.SqlServer

# Install a specific version
dotnet add package Microsoft.EntityFrameworkCore --version 6.0.0

# Install ASP.NET Core Identity (for authentication)
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore
```

Updating Packages

Using Visual Studio:

- Open Manage NuGet Packages → click the "Updates" tab
- Select packages with available updates → click "Update"

Using .NET CLI:

```
# Update a package to the latest version
dotnet add package Microsoft.EntityFrameworkCore --version 7.0.0

# List all installed packages (to check current versions)
dotnet list package

# Remove a package
dotnet remove package Microsoft.EntityFrameworkCore.SqlServer

# Restore all missing packages (run after cloning a project)
dotnet restore
```

Package Reference in .csproj File

When a package is installed, Visual Studio or the CLI automatically records it in the project file (.csproj). This allows any developer to restore all packages on a new machine using dotnet restore:

```
<!-- StudentManagement.csproj -->
<ItemGroup>
  <PackageReference
    Include="Microsoft.EntityFrameworkCore.SqlServer"
    Version="6.0.0" />
  <PackageReference
    Include="Microsoft.AspNetCore.Identity.EntityFrameworkCore"
    Version="6.0.0" />
</ItemGroup>
```

Commonly Used NuGet Packages in ASP.NET Core

Package Name	Purpose
Microsoft.EntityFrameworkCore.SqlServer	EF Core with SQL Server database support
Microsoft.AspNetCore.Identity.EntityFrameworkCore	User authentication and management
Microsoft.AspNetCore.Authentication.JwtBearer	JWT token-based authentication for APIs
Newtonsoft.Json	Advanced JSON serialization and deserialization
Serilog.AspNetCore	Structured, advanced logging framework
Swashbuckle.AspNetCore	Swagger UI for documenting Web APIs

Reference: Freeman, A. (2022). *Pro ASP.NET Core 6 (9th ed.)*. Apress. — Chapter 4 (Managing Packages)

Mulund College of Commerce (Autonomous)

Mulund (W), Mumbai — Affiliated to University of Mumbai | PTVA

T.Y. B.Sc. Information Technology | Semester V

Subject: ASP.NET Core

Assignment 2

Due Date:	26th July 2026	Max Marks:	—
Reference:	Pro ASP.NET Core 6 — Adam Freeman (Apress, 9th Ed.)	Class:	T.Y. B.Sc. IT — Sem V

- Q** Create an ASP.NET Core web application with navigation controls and a shopping cart for product browsing and customer selection management. Explain the development steps.
- 1.**

Answer

Overview

In this answer, we build the SportsStore application — a real-world Blazor-based eCommerce application from Pro ASP.NET Core 6 by Adam Freeman. The application demonstrates navigation controls for category-based product browsing and a complete shopping cart for customer selection management.

Step 1 — Create the Project

```
# Create a new Blazor Server project
dotnet new blazorserver -n SportsStore
cd SportsStore
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Step 2 — Define the Data Model

Create the Product model class representing items for sale in the store:

```
// Models/Product.cs
namespace SportsStore.Models {
    public class Product {
        public long ProductId { get; set; }
        public string Name { get; set; } = string.Empty;
        public string Description { get; set; } = string.Empty;
        public decimal Price { get; set; }
        public string Category { get; set; } = string.Empty;
    }
}
```

Step 3 — Set Up the Database Context

```
// Models/StoreDbContext.cs
using Microsoft.EntityFrameworkCore;

public class StoreDbContext : DbContext {
    public StoreDbContext(DbContextOptions<StoreDbContext> options)
        : base(options) { }

    public DbSet<Product> Products => Set<Product>();
}

// Program.cs – Register database context
builder.Services.AddDbContext<StoreDbContext>(opts =>
    opts.UseSqlServer(
        builder.Configuration["ConnectionStrings:SportsStoreConnection"]));
```

Step 4 — Add Navigation Controls

Navigation controls allow users to filter products by category. A NavLink component is created in Blazor to display available categories and highlight the selected one:

```
<!-- Pages/Store.razor – Navigation with category filter -->
@inject IRepository Repository

<div class="container-fluid">
    <div class="row">
        <!-- Navigation Panel (Left Column) -->
        <div class="col-3">
            <NavLink class="btn btn-outline-dark" href="/store">All</NavLink>
            @foreach (string category in Categories) {
                <NavLink class="btn btn-outline-secondary"
                    href="@($" /store/{category}")">
                    @category
                </NavLink>
            }
        </div>
        <!-- Product List (Right Column) -->
        <div class="col-9">
            @foreach (Product p in ProductData) {
                <ProductSummary Product="@p" />
            }
        </div>
    </div>
</div>

@code {
    public IEnumerable<string> Categories =>
        Repository.Products.Select(p => p.Category).Distinct().OrderBy(c => c);

    public IEnumerable<Product> ProductData =>
        SelectedCategory == null
            ? Repository.Products
            : Repository.Products.Where(p => p.Category == SelectedCategory);

    [Parameter] public string? SelectedCategory { get; set; }
}
```

Step 5 — Build the Shopping Cart Model

The shopping cart stores items selected by the customer. It is implemented as a class with methods to add, remove, and calculate totals:

```
// Models/Cart.cs
public class Cart {
    private List<CartLine> Lines = new List<CartLine>();

    public IEnumerable<CartLine> CartLines => Lines;

    public void AddItem(Product product, int quantity) {
        CartLine? line = Lines
            .FirstOrDefault(l => l.Product.ProductId == product.ProductId);
        if (line == null) {
            Lines.Add(new CartLine { Product = product, Quantity = quantity });
        } else {
            line.Quantity += quantity;
        }
    }

    public void RemoveLine(Product product) =>
        Lines.RemoveAll(l => l.Product.ProductId == product.ProductId);

    public decimal TotalValue() =>
        Lines.Sum(l => l.Product.Price * l.Quantity);

    public void Clear() => Lines.Clear();
}

public class CartLine {
    public int CartLineId { get; set; }
    public Product Product { get; set; } = new();
    public int Quantity { get; set; }
}
```

Step 6 — Register Cart as a Scoped Service

The Cart is registered with the DI container as a Scoped service so each user session gets its own cart instance:

```
// Program.cs
builder.Services.AddScoped<Cart>();
```

Step 7 — Create the Cart Component (UI)

```

<!-- Pages/CartPage.razor -->
@inject Cart Cart
@inject NavigationManager NavManager

<h2>Your Shopping Cart</h2>

<table class="table table-bordered">
    <thead>
        <tr>
            <th>Product</th>
            <th>Qty</th>
            <th>Price</th>
            <th>Subtotal</th>
            <th>Action</th>
        </tr>
    </thead>
    <tbody>
        @foreach (CartLine line in Cart.CartLines) {
            <tr>
                <td>@line.Product.Name</td>
                <td>@line.Quantity</td>
                <td>@line.Product.Price.ToString("c")</td>
                <td>@((line.Product.Price * line.Quantity).ToString("c"))</td>
                <td>
                    <button class="btn btn-danger btn-sm"
                        @onclick="@(() => RemoveItem(line.Product))">
                        Remove
                    </button>
                </td>
            </tr>
        }
    </tbody>
    <tfoot>
        <tr>
            <td colspan="3"><b>Total:</b></td>
            <td>@Cart.TotalValue().ToString("c")</td>
            <td></td>
        </tr>
    </tfoot>
</table>

<button class="btn btn-secondary" @onclick="ContinueShopping">
    Continue Shopping
</button>

@code {
    void RemoveItem(Product p) => Cart.RemoveLine(p);
    void ContinueShopping() => NavManager.NavigateTo("/store");
}

```

Development Steps — Summary

Step	Task	Key Files
1	Create Blazor Server project	SportsStore.csproj

Step	Task	Key Files
2	Define Product data model	Models/Product.cs
3	Set up EF Core database context	Models/StoreDbContext.cs, Program.cs
4	Add navigation controls (categories)	Pages/Store.razor
5	Build Cart model (Add/Remove/Total)	Models/Cart.cs, CartLine.cs
6	Register Cart as Scoped service	Program.cs
7	Create Cart UI component	Pages/CartPage.razor

Reference: Freeman, A. (2022). *Pro ASP.NET Core 6* (9th ed.). Apress. — Chapters 8 & 9 (*SportsStore: Navigation and Cart*)

Q Apply the steps to create an ASP.NET Core application by adding data, displaying a list of products, implementing pagination, and styling the application.

Answer

Overview

This answer demonstrates the process of building the data layer of the SportsStore application — from creating the project and adding seed data, to displaying a paginated product list with Bootstrap styling. This follows Chapter 7 of *Pro ASP.NET Core 6* by Adam Freeman.

Step 1 — Create the Project

```
dotnet new blazorserver -n SportsStore
cd SportsStore
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Step 2 — Add the Data Model

The Product class is the core data model. Each product has an Id, Name, Description, Price, and Category:

```
// Models/Product.cs
namespace SportsStore.Models {
    public class Product {
        public long ProductId { get; set; }
        public string Name { get; set; } = string.Empty;
        public string Description { get; set; } = string.Empty;
        public decimal Price { get; set; }
        public string Category { get; set; } = string.Empty;
    }
}
```

The StoreDbContext bridges the Product model and the database:

```
// Models/StoreDbContext.cs
public class StoreDbContext : DbContext {
    public StoreDbContext(DbContextOptions<StoreDbContext> opts) : base(opts) { }
    public DbSet<Product> Products => Set<Product>();
}
```

Step 3 — Seed the Database with Initial Data

Seed data provides sample products to display in the application without needing to manually insert records:

```
// Models/SeedData.cs
public static class SeedData {
    public static void SeedDatabase(StoreDbContext context) {
        context.Database.Migrate();
        if (!context.Products.Any()) {
            context.Products.AddRange(
                new Product { Name="Kayak", Price=275M, Category="Watersports",
                    Description="A boat for one person" },
                new Product { Name="Lifejacket", Price=48.95M, Category="Watersports",
                    Description="Protective and fashionable" },
                new Product { Name="Soccer Ball", Price=19.50M, Category="Soccer",
                    Description="FIFA-approved size and weight" },
                new Product { Name="Corner Flags", Price=34.95M, Category="Soccer",
                    Description="Give your pitch a professional touch" },
                new Product { Name="Stadium", Price=79500M, Category="Soccer",
                    Description="Flat-packed 35,000-seat stadium" }
            );
            context.SaveChanges();
        }
    }
}

// Call in Program.cs:
SeedData.SeedDatabase(app.Services.CreateScope()
    .ServiceProvider.GetRequiredService<StoreDbContext>());
```

Step 4 — Display the Product List

Create a Blazor component to display products. A ProductSummary component shows each product's name, description, and price with an "Add to Cart" button:

```

<!-- Shared/ProductSummary.razor -->
@inject Cart Cart

<div class="card m-2">
    <div class="card-body">
        <h5 class="card-title">@Product.Name</h5>
        <p class="card-text">@Product.Description</p>
        <p class="card-text"><b>@Product.Price.ToString("c")</b></p>
        <button class="btn btn-primary"
            @onclick="@(() => Cart.AddItem(Product, 1))">
            Add to Cart
        </button>
    </div>
</div>

@code {
    [Parameter] public Product Product { get; set; } = new();
}

```

Step 5 — Implement Pagination

Pagination limits the number of products displayed per page. A `PagingData` class tracks pagination state, and navigation buttons allow users to move between pages:

```
// Models/PagingData.cs
public class PagingData {
    public int CurrentPage { get; set; } = 1;
    public int PageSize { get; set; } = 4;
    public int TotalPages { get; set; }

    public bool PreviousPage => CurrentPage > 1;
    public bool NextPage => CurrentPage < TotalPages;
}

<!-- Store.razor – Applying pagination to product list -->
@code {
    public PagingData Paging { get; set; } = new();

    public IEnumerable<Product> ProductData =>
        Repository.Products
            .Skip((Paging.CurrentPage - 1) * Paging.PageSize)
            .Take(Paging.PageSize);

    protected override void OnParametersSet() {
        Paging.TotalPages = (int)Math.Ceiling(
            (decimal)Repository.Products.Count() / Paging.PageSize);
    }
}

<!-- Pagination Navigation Buttons -->
<div class="d-flex justify-content-between mt-3">
    <button class="btn btn-secondary" disabled="@(!Paging.PreviousPage)"
        @onclick="@(() => Paging.CurrentPage--)">Previous</button>
    <span>Page @Paging.CurrentPage of @Paging.TotalPages</span>
    <button class="btn btn-secondary" disabled="@(!Paging.NextPage)"
        @onclick="@(() => Paging.CurrentPage++)">Next</button>
</div>
```

Step 6 — Style the Application with Bootstrap

Bootstrap is the default CSS framework included with ASP.NET Core Blazor projects. It provides ready-made CSS classes for responsive, consistent styling:

Bootstrap Class	Applied To	Effect
container-fluid	Outer layout div	Full-width responsive container
row / col-3 / col-9	Layout grid	Sidebar (nav) + main content (products)
card / card-body	Product items	Boxed product cards with padding
btn btn-primary	Add to Cart button	Blue styled action button
btn btn-outline-dark	Navigation links	Outlined category filter buttons
table table-bordered	Cart page table	Clean bordered table layout
d-flex justify-content-between	Pagination bar	Evenly spaced Previous/Next buttons

```

<!-- _Layout.cshtml – Link Bootstrap CSS -->
<link rel="stylesheet"
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" />

<!-- Example styled product grid -->
<div class="container-fluid">
    <div class="row">
        <div class="col-3 bg-light p-3">
            <!-- Navigation panel -->
        </div>
        <div class="col-9">
            <div class="row row-cols-1 row-cols-md-3 g-4">
                @foreach (Product p in ProductData) {
                    <div class="col">
                        <ProductSummary Product="@p" />
                    </div>
                }
            </div>
        </div>
    </div>
</div>

```

Complete Development Steps — Summary

Step	Task	Outcome
1	Create Blazor Server project + install EF Core packages	Runnable project scaffold with correct dependencies
2	Add Product model and StoreDbContext	Data layer ready — C# class maps to database table
3	Seed database with sample products	Application has data to display on first run
4	Create ProductSummary component to display product list	Products shown as cards with name, description, price
5	Implement PagingData class and apply Skip/Take to product query	Products split across pages — 4 per page by default
6	Add Bootstrap classes to layout and components	Professional, responsive UI with grid layout and styled buttons

Reference: Freeman, A. (2022). *Pro ASP.NET Core 6* (9th ed.). Apress. — Chapter 7 (*SportsStore: A Real Application*)